# Advanced Topics in Databases: Final Report

Amos Uwamusi up202502075 Kamil Tasarz up202512958
Sérgio Cardoso up202107918

June 4, 2026

## 1 Introduction

This report describes our work for the Advanced Topics in Databases course at FCUP: an election analytics platform for Portuguese local (*Autárquicas*) elections. Official results from the Comissão Nacional de Eleições (CNE) arrive as multi-sheet Excel workbooks; administrative boundaries come from DGT CAOP. Our goal was a database-centred system rather than a spreadsheet workflow: persist the data in PostgreSQL with PostGIS, load it through a rerunnable ETL, implement non-trivial server-side SQL (including D'Hondt seat allocation, window functions, `ROLLUP`, and `CUBE`), and expose outcomes through a thin Flask web client that queries the database directly with psycopg2.

The implemented load covers Câmara Municipal results at municipality level for the 2017 and 2021 election years, with sample validation against CNE figures for Lisboa, Porto, and Barrancos. The following sections document the operational and warehouse schema design, the ETL and cleaning decisions, PL/pgSQL functions and triggers, analytical queries, the web frontend, known limitations, and each member's contribution.

## 2 Development

### 2.1 Problem scope and chosen election datasets

Portuguese local election results are distributed as multi-sheet CNE workbooks. Lists are often identified by local codes (e.g. A, B in Lisboa 2021), not only national party acronyms. Vote counts, turnout, and mandates appear in different sheets; boundary files use another naming scheme. The project goal is a database-centred application: store data in a normalized schema and a warehouse, load it through a rerunnable ETL, expose non-trivial SQL (D'Hondt, window functions, ROLLUP, CUBE), and serve a thin web UI, in this case using Flask, that queries PostgreSQL with psycopg2 (no ORM).
So, for this project, we chose to focus on the 2021 and 2017 local elections.

- **2021 Local Elections:** This dataset includes detailed results from the 2021 local elections in Portugal, covering various municipalities and parties. It provides a comprehensive view of the electoral landscape during that period.
- **2017 Local Elections:** This dataset contains results from the 2017 local elections, allowing us to analyze trends and changes in voter behavior and party performance over time.
- **DGT CAOP 2021:** This dataset includes boundary files for the municipalities, which are essential for spatial analysis and visualization of election results.

### 2.2 Operational Schema and Warehouse Design Rationale

The system uses three PostgreSQL schemas: staging (`sql/05_staging_schema.sql`), operational (`sql/01_operational_schema.sql`), and warehouse (`sql/02_warehouse_schema.sql`).
The following ER diagrams (obtained with pgAdmin's ERD tool) illustrate the design of the operational and warehouse schemas, highlighting the relationships between tables and the overall structure of the database. The operational schema is designed to store raw and processed data for day-to-day operations, while the warehouse schema is optimized for analytical queries and reporting.

**Keys and indexing.** Primary and foreign keys enforce election–organ–territory–party integrity; `candidacy` is the hub linking lists to `vote_result`, `seat_result`, and related facts. B-tree indexes on

`candidacy` (election, organ, municipality) and on `vote_result` (candidacy, votes descending) match the usual filters (one contest per municipality, top parties by votes). GiST indexes on district and municipality `geometry` support map bounding-box queries. Warehouse indexes on fact and dimension keys speed star-schema joins; `party_municipality_summary` is indexed by election, municipality, and party for quick lookups after `refresh_party_municipality_summary()`.

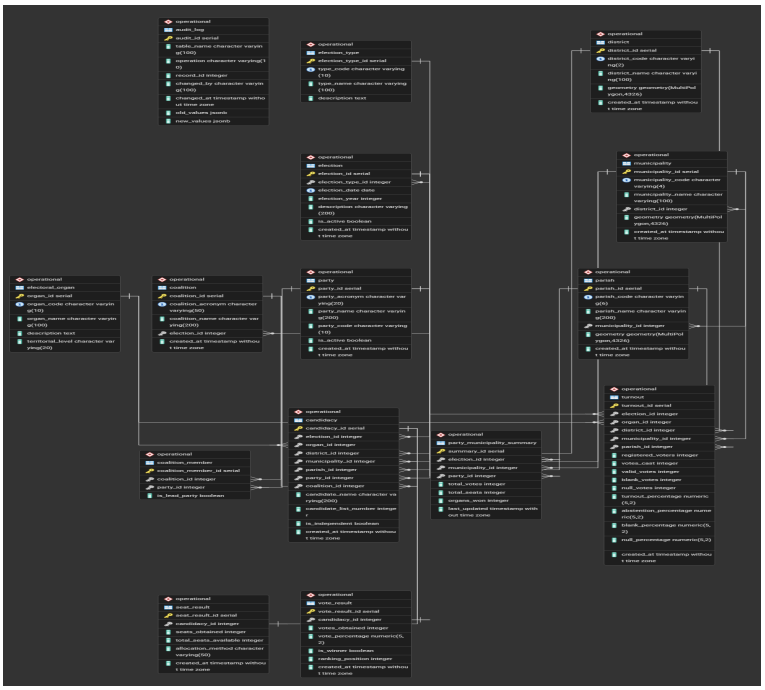

Figure 1: Operational Schema ER Diagram

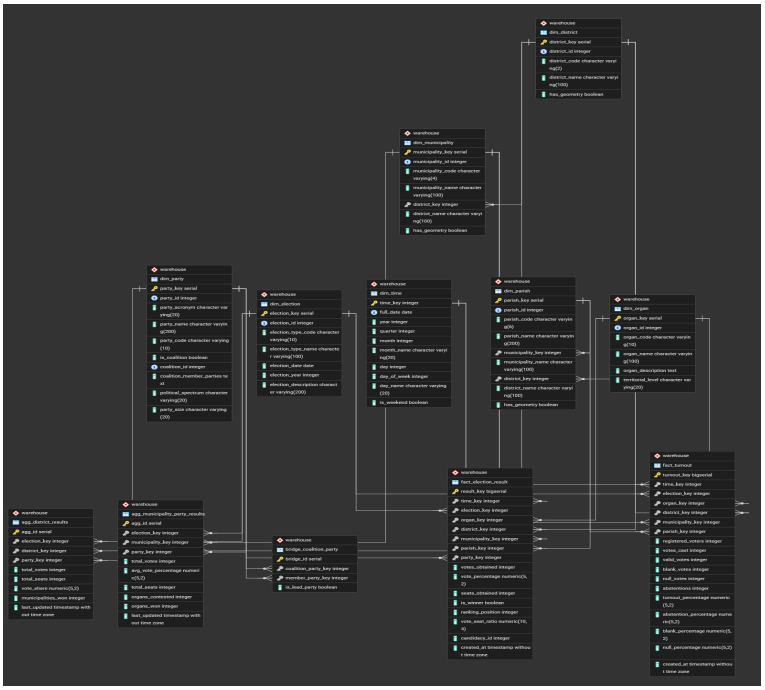

Figure 2: Warehouse Star Schema ER Diagram

## 2.3 ETL Pipeline

The Python ETL (`etl/run_etl.py`, `--mode full`) loads **Câmara Municipal (CM)** results at **municipality** level for elections `aut_2017` and `aut_2021`. Votes and turnout come from CNE **mapa_1** (wide Excel, unpivoted in `pipeline/extract.py` into `staging.stg_election_results` and `stg_turnout_data`); official seat counts from **mapa_2** (`pipeline/load_seats.py` → `seat_result`). District and municipality boundaries are joined from DGT CAOP via `pipeline/transform_geo.py`. Each run is logged in `staging.stg_etl_run_log` and can be repeated per `election_id`.

| Phase | Module | Main output |
|---|---|---|
| Extract | `extract.py` | `stg_election_results`, `stg_turnout_data` |
| Operational | `transform_operational.py` | parties, `candidacy`, `vote_result`, `turnout` |
| Geometries | `transform_geo.py` | PostGIS `geometry` on districts/municipalities |
| Warehouse | `load_warehouse.py` | `dim_*`, `fact_*` |
| Seats | `load_seats.py` | `seat_result` (CNE mandates) |
| Post-load | `post_load.py` | `party_municipality_summary`, turnout % |

**Cleaning and load policy.** Wide sheets are unpivoted to one row per list column; votes are coerced to integers; territory names are normalized (title case); only CM rows are kept (empty parish field). Parties are resolved with `get_or_create_party`. Municipalities are matched to CAOP by **CNE names**, not a single official concelho code end-to-end. During bulk insert, `session_replication_role = replica` disables audit triggers; vote and turnout percentages are recomputed in post-load (`calculate_turnout_percentages`, `refresh_party_municipality_summary`).

**Validation.** Spot checks for Lisboa, Porto, and Barrancos (2021) show zero mismatches against CNE for votes, turnout, percentages, and seats. About 50 municipalities appear in `mapa_2` without corresponding `mapa_1` rows; they remain without vote facts in the database (documented source gap, not an ETL bug).

## 2.4 Functions, Views, and Triggers

All functions, views, and triggers were implemented in PostgreSQL using PL/pgSQL.

- **Functions:**
  - `calculate_vote_percentage (p_candidacy_id)`: Calculate vote percentage for a candidacy.
  - `get_party_performance_in_municipality (p_election_id, p_municipality_id, p_party_acronym)`: Get the performance of a party in a specific municipality.
  - `get_top_parties (p_election_id, p_municipality_id, p_organ_id, p_limit)`: Get the top parties in a specific municipality.
  - `allocate_seats_dhondt (p_election_id, p_organ_id, p_municipality_id, p_total_seats)`: Allocate seats based on the D'Hondt method for a specific election and municipality.
  - `calculate_turnout_percentages ()`: Batch recompute turnout and related percentages after ETL.
  - `get_or_create_party (p_party_acronym, p_party_name)`: Get or create a party based on its acronym and name.
- **Views:**
  - `vw_complete_results`: A view that provides complete election results, including vote counts, percentages, and seat allocations for each candidacy.
  - `vw_turnout_analysis`: A view that provides turnout analysis for each election and municipality.
  - `vw_candidacy_details`: A view that provides complete candidacy information, including vote percentages and seat allocations.
  - `vw_municipality_summary`: A view that summarizes election results by municipality, including total votes, turnout, and seat allocations.
  - `vw_stg_missing_data`: A view that identifies any missing data in the staging tables, such as missing votes or candidacy information.

  - `vw_stg_potential_duplicates`: A view that identifies potential duplicate records in the staging tables, such as duplicate votes or candidacies.

- **Triggers:**
  - `trg_turnout_percentages` (on `turnout`): before insert/update, derives turnout, abstention, blank, and null percentages from registered voters and votes cast.
  - `trg_vote_percentages` (on `vote_result`): before insert/update, sets `vote_percentage` from the contest valid-vote total in `turnout`.
  - `trg_audit_candidacy` and `trg_audit_vote_result`: after changes on candidacy or vote rows, append rows to `audit_log` via `trg_audit_log()`. Bulk ETL disables triggers; percentages and summaries are refreshed in post-load.

## 2.5 Analytical Queries

Beyond operational functions and triggers, we added eight analytical views and `demonstrate_dhondt` in `sql/04_analytical_queries.sql`. They are applied **after** ETL (not part of the load). The Flask UI does **not** query these objects: pages and APIs use operational tables and simpler warehouse/operational views (`vw_candidacy_details`, `vw_municipality_summary`, map endpoints) for municipality-level browsing, maps, and 2017 vs 2021 charts. Results from `ROLLUP`, `CUBE`, and district-wide ranking views are wide, hierarchical tables that would need separate report-style screens or exports; we kept them in PostgreSQL to satisfy the advanced SQL requirements and document findings without extending the web layer. They are meant for direct use in `psql`/pgAdmin or via `scripts/run_sql_demos.py`, which writes `docs/sql_outputs/demo_results.txt`.

| View / function | SQL technique |
|---|---|
| `analytical_query_1_party_rankings` | `RANK`, running totals |
| `analytical_query_2_district_comparison` | Party % vs district average |
| `analytical_query_3_turnout_analysis` | `NTILE`, quartiles |
| `analytical_query_4_rollup_hierarchical` | `GROUP BY ROLLUP` |
| `analytical_query_5_cube_multidimensional` | `GROUP BY CUBE` |
| `analytical_query_6_advanced_aggregates` | `FILTER`, `STRING_AGG`, `ARRAY_AGG` |
| `analytical_query_8_cross_district_comparison` | Cross-district ranks |
| `demonstrate_dhondt` | Ranked D'Hondt quotients |

**Key findings (CM, validated against CNE samples).**

- **Lisboa 2021:** lists **A** (35.35%) and **B** (34.38%) lead; local codes matter more than a single national-party label (`calculate_vote_percentage` matches stored %).
- **Cross-year Lisboa:** 2017 winner **PS** (43.89%) vs 2021 winner **A** (35.35%) — same municipality, different list coding between elections.
- **District contrast (2021):** Lisboa list A is +14.87 percentage points above its district average; PSD is −20.74 pp (`analytical_query_2`).
- **Seats:** CNE `mapa_2` gives A/B seven mandates each in a 17-seat council; `allocate_seats_dhondt` with **7** seats is an illustrative run (3+3+1), separate from official allocation.
- **Porto D'Hondt (7 seats):** PS 5, B 2 (`demonstrate_dhondt`, `run_sql_demos.py`).
- **Warehouse scale:** after full load, ∼1 256 `fact_election_result` and ∼281 `fact_turnout` rows (see demo output tail).

## 2.6 Frontend Overview

The web client (`app/app.py`) is a thin **Flask** application that queries PostgreSQL with **psycopg2** and raw SQL—no ORM. The user selects an election year from the navbar (`election_id` query parameter); all pages and chart APIs scope results to that election.

**Pages and data flow.** `/` shows national summary statistics and a Leaflet map of districts (GeoJSON from `/api/map/districts`; click opens `/district/<id>`). The districts page lists all concelhos with average turnout and a comparison chart. `/municipality/<id>` renders CM results as an HTML table

plus Chart.js bar charts (`/api/charts/party_comparison`). `/analytics` compares **2017 vs 2021** via `/api/charts/election_comparison`, Chart.js national charts, and server-rendered Matplotlib PNGs (`/analytics/chart.png`, built in `app/charts.py`; static copies via `scripts/export_charts.py`).

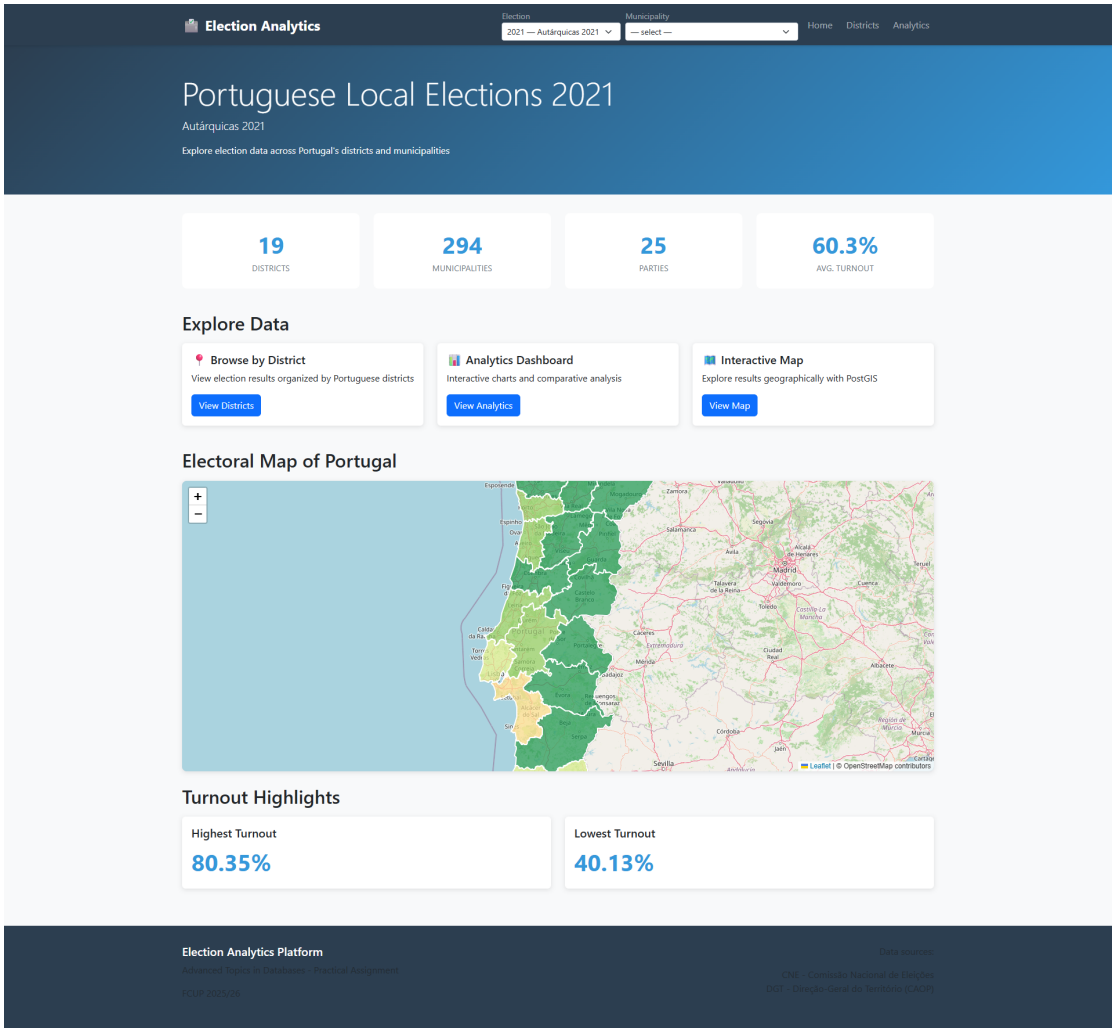


Figure 3: Home page: election selector, summary cards, Leaflet district map (PostGIS geometries), and turnout highlights.

The UI deliberately uses operational/warehouse views and tailored API queries rather than the `analytical_query_*` views (see previous subsection). Run locally with `python app/app.py` on port 8000 after ETL (`docs/reproducibility.md`).

## 2.7 Known Limitations and Possible Extensions

**Current scope and gaps.**

- **Elections and organ:** only Autárquicas **2017** and **2021**, organ **Câmara Municipal (CM)**, **municipality** level. Not loaded: `mapa_3` (elected persons), Assembleia Municipal, Junta de Freguesia, or parish-level facts (parish exists in DDL only).
- **Party identity:** lists use **local codes** (e.g. A/B in Lisboa 2021); cross-year or national party comparison is misleading without manual mapping.
- **Source gaps:** ∼50 municipalities appear in CNE `mapa_2` without matching `mapa_1` rows, so they have seat metadata but no vote facts in the database.
- **Geography:** CAOP fallback GeoJSON covers the **continent**; island municipalities may lack `geometry` until full official shapefiles are added.

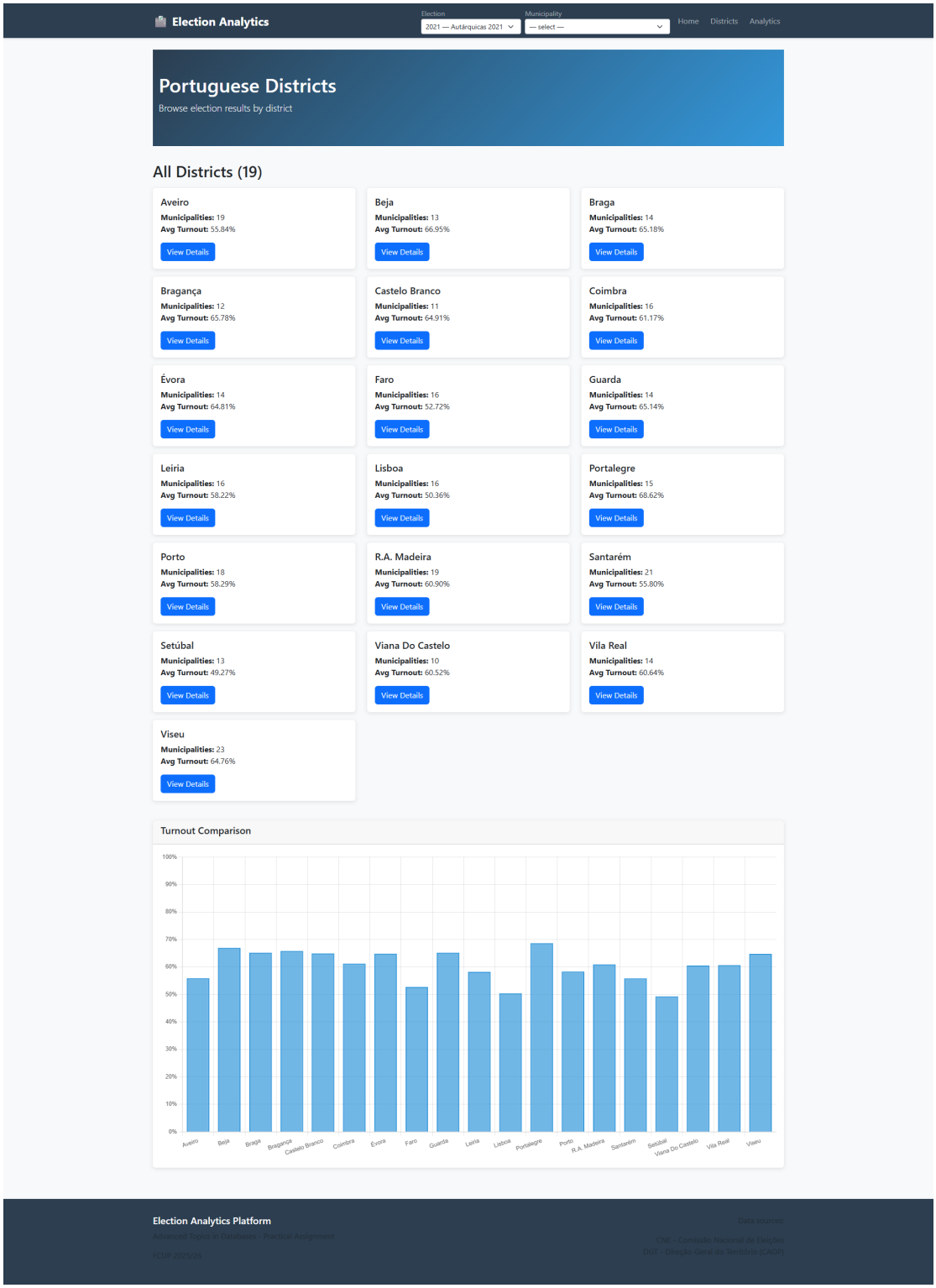


Figure 4: Districts page: per-district municipality counts and turnout, with Chart.js comparison across districts.

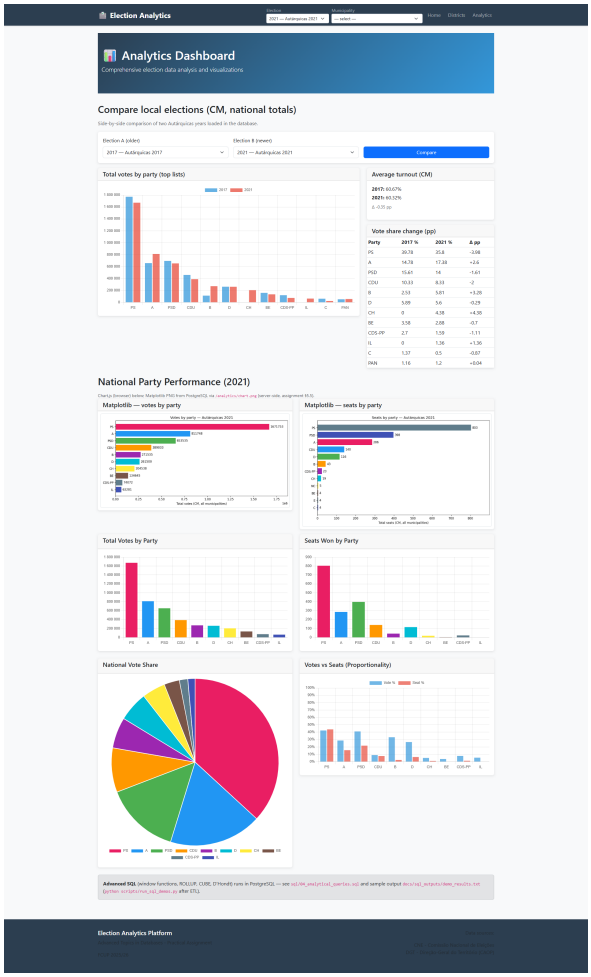


Figure 5: Analytics dashboard: 2017 vs 2021 comparison, Matplotlib charts from PostgreSQL, and Chart.js vote/seat visualisations.

- **Warehouse:** tables `warehouse.agg_*` are defined but **not populated** in the MVP pipeline.
- **Operations:** no user authentication; ETL is **batch** only (not live election-night ingestion). Turnout percentages require post-load refresh on older runs (`post_load.py` or `scripts/fix_turnout_percentages.py`) because bulk load disables triggers.
- **Web vs SQL:** advanced `analytical_query_*` views are not exposed in the Flask UI (see Analytical Queries).

**Possible extensions.**

- Populate `agg_*`, add AM/JF organs and parish-level ETL; load additional election years into `DATASETS`.
- Full CAOP (including islands); richer map UX (e.g. AJAX detail panel) or Plotly dashboards.
- Wire selected analytical views into dedicated report/export pages; national-party normalization table for cross-municipality comparison.

### 2.8 Short Statement of Individual Contributions

**Amos Uwamusi up202502075**
Designed the **operational, staging, and warehouse database schemas**; implemented **SQL functions, triggers, and analytical queries** including D'Hondt allocation; built the first version of the **Flask web application** and page templates; developed the **initial ETL pipeline** and core project documentation; corrected analytical query definitions after testing.

**Kamil Tasarz up202512958**
Refactored the ETL into **modular pipeline stages** and full-load orchestration; added loading of **official seat results** and **Autárquicas 2017** data for cross-year use; performed **CNE validation** and wrote reconciliation and reproducibility documentation; exported **ER diagrams** and prepared SQL demonstration outputs for the report; extended the web application with **analytics charts** and cross-election comparison APIs; fixed data-quality issues in turnout, seats, and charts; consolidated the project README and task list.

**Sérgio Cardoso up202107918**
Obtained and organised **CNE election datasets** and Python dependencies for the team; contributed to early ETL configuration and schema setup alongside the data files; authored and maintained the **LaTeX final report** and **Beamer presentation slides**; updated report and slide content as the implementation evolved.

## 3 Conclusion

We delivered a **database-centred** election analytics platform for Portuguese *Autárquicas* (2017 and 2021), aligned with the TABD project goals:

- **Data and ETL:** CNE `mapa_1`/`mapa_2` and DGT CAOP integrated through a rerunnable Python pipeline (staging → operational → warehouse), with spot validation (Lisboa, Porto, Barrancos) showing no mismatches against official figures.
- **Database design:** normalized operational schema (`candidacy` hub, PostGIS territories) and a star-schema warehouse; PL/pgSQL functions, triggers, audit log, and D'Hondt allocation, with vote/turnout percentages refreshed after bulk load.
- **Advanced SQL:** analytical views using window functions, `ROLLUP`, and `CUBE`, documented via `run_sql_demos.py` and `demo_results.txt` (e.g. Lisboa 2021 lists A/B leading at 35.35% / 34.38%).
- **Web client:** thin Flask application (psycopg2, no ORM) with Leaflet maps, Chart.js municipality charts, and 2017 vs 2021 analytics including Matplotlib exports.
- **Limitations:** MVP scope is CM at municipality level only; local list codes, continent CAOP fallback, empty `agg_*`, and analytical views not wired into the UI remain documented for future work.

Reproducibility steps for evaluators are in `docs/reproducibility.md`.

# A References


- Comissão Nacional de Eleições (CNE). *Eleições Autárquicas 2021* — official results spreadsheets (`mapa_1`, `mapa_2`). `https://www.cne.pt/content/eleicoes-autarquicas-2021`
- Direção-Geral do Território (DGT). *Cartografia Administrativa Oficial de Portugal (CAOP)*. `https://www.dgterritorio.gov.pt/atividades/cartografia/cartografia-tematica/caop`
- nmota. *caop_GeoJSON* — GeoJSON fallback for CAOP boundaries when the official archive is unavailable. `https://github.com/nmota/caop_GeoJSON`
- PostgreSQL Global Development Group. *PostgreSQL Documentation*; PostGIS Project. *PostGIS Documentation*.
- Python Software Foundation. *Python*; Psycopg contributors. *psycopg2*.
- Pallets Projects. *Flask*; Leaflet contributors. *Leaflet*; Chart.js contributors. *Chart.js*; Matplotlib development team. *Matplotlib*.
- FCUP — *Advanced Topics in Databases* (2025/26): course lectures, lab materials, and practical assignment guidelines.
- Anysphere. *Cursor IDE* — AI-assisted editor used for parts of the implementation and documentation. `https://cursor.com`